

Addressing the 4+1 Software Safety Assurance Principles within Scrum

Osama Doss

Department of Computer Science
University of York
York, United Kingdom
Telephone: +44 1904 325464
osad500@york.ac.uk

Tim Kelly

Department of Computer Science
University of York
York, United Kingdom
Telephone: +44 1904 325477
Tim.kelly@york.ac.uk

ABSTRACT

The 4+1 principles document common principles of software safety assurance that can be observed from software safety standards and best practice. These principles are constant across domains and across projects, and can be regarded as the immutable core of any software safety justification. The principles also help maintain understanding of the 'big picture' of software safety issues whilst examining and negotiating the detail of individual standards, and provide a reference model for cross-sector certification. This document discusses the challenges posed when applying these principles to Scrum, and makes initial recommendations for how the principles can be accommodated within a Scrum development.

Categories and Subject Descriptors

D.3.3 [Software Engineering]: Design – Methodologies

General Terms

Management, Theory, Documentation, Reliability, Verification.

Keywords

Keywords: Scrum. Safety. Assurance. Certification. Assurance Case. Software Safety. 4+1 principles

1. INTRODUCTION

Douglass [6,8] highlighted some of the main challenges posed by the use of agile methods for safety critical systems:

1. Agile de-emphasizes documentation but safety standards require strong documentation
2. Agile literature has no discussion of safety, reliability, or security
3. Agile requirements are typically very lightweight with no traceability
4. Agile testing is continuous but lightweight
5. Use tooling to automatically generate documentation for certification approval
6. Requirements must be specified, reviewed and mapped to design, tests and code
7. Process steps must include dependability analysis and assessment
8. Process must be documented, with standards (checklists) that

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for components of this work owned by others than ACM must be honored. Abstracting with credit is permitted. To copy otherwise, or republish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee. Request permissions from Permissions@acm.org.
XP '16 Workshops, May 24 2016, Edinburgh, Scotland UK
© 2016 ACM. ISBN 978-1-4503-4134-9/16/05...\$15.00
DOI: <http://dx.doi.org/10.1145/2962695.2962712>

can be verified by QA

Agile development traditionally promotes the view that working software is the primary measure of progress [7]. However, for safety critical systems development the definition of 'working' software needs to be safety *assured* software. If the safety of the software isn't assured, it cannot be considered working, because it cannot be used.

Therefore our primary measures of progress (for safety critical software) must take into account both the status of the software and the status of software safety assurance.

Scrum is an agile development approach that uses an iterative, incremental process, and is based on a small set of core values, principles, and practices [1]. Scrum does not explicitly address software safety assurance. In this paper we discuss some of the most significant challenges posed by attempting to integrate software safety assurance into Scrum.

This paper is organised as follows: Section 2 presents the 4+1[5] Software Safety Assurance Principles used to describe the typical assurance requirements of software safety standards. Section 3 discusses the challenges posed by addressing these principles within Scrum, and recommends possible ways forward for adapting Scrum to better address the principles. Section 4 discusses the further work required to validate these proposals. Section 5 presents conclusions.

2. 4+1 Fundamental Principles of Software Safety Assurance

There are many standards that either directly or indirectly address software safety assurance (e.g. IEC 61508, ISO 26262, EN 50128, parts of UK Defence Standard 00-56, and DO-178B). There are easily observed differences in the details of these standards. For example, DO-178B uses the concept of *Development Assurance Levels* (DALs) to moderate the *objectives* of the standard according to the criticality of the software under development, whereas IEC 61508 uses the concept of *Safety Integrity Levels* (SILs) to make recommendations as to suitable design and assurance techniques according to the criticality of the software under development. The requirements and recommendations for SILs in IEC 61508 are not the same as those for DALs in DO-178B. However, underlying these differences there is a number of fundamental principles that can be observed in many of the current standards. The following five assurance principles, previously presented in [2], can be identified:

- **Principle 1:** Software safety requirements shall be defined to address the software contribution to system hazards
- **Principle 2:** The intent of the software safety requirements shall be maintained throughout requirements decomposition

- **Principle 3:** Software safety requirements shall be satisfied
- **Principle 4:** Hazardous behaviour of the software shall be identified and mitigated
- **Principle 4+1** The confidence established in addressing the software safety principles shall be commensurate to the contribution of the software to system risk

3. Challenges and Recommendations for Scrum

3.1 Software Safety Assurance Principle 1: Software safety requirements shall be defined to address the software contribution to system hazards

Software by itself cannot be safe or unsafe. It is only when placed in a system context that the ‘safety’ of software can be judged by considering the contribution that the software could make to system level hazards. For example, software can play a role as the initiator of a causal chain of events leading to a system level hazard (and eventual accident). Software (e.g. when placed in the role of a protection system) can also play a role by failing to mitigate failures of other (non-software) elements. The first challenge of software safety assurance is to identify all of the ways in which software can contribute to system level hazards and to capture the necessary behaviour of the software in relation to these contributions in terms of a clearly defined set of software safety requirements at the system software boundary. (Note these behavioural software safety requirements are distinct from the assurance requirements of software safety standards, e.g. to maintain traceability.)

Challenges posed

- For assurance (e.g. certification), software safety requirements (and their treatment) must be easily identifiable from amongst the total set of requirements being addressed
- Safety requirements can be positive functional requirements in their own right, constraints to be satisfied (e.g. timing) when addressing other requirements, or negative requirements (i.e. behavior to not be exhibited)
- Safety requirements are therefore often associated and interrelated with core product requirements (i.e. requirements in the product backlog)
- Safety requirements should not be addressed as an ‘after thought’ following implementation of core product requirements
- Relevant (related) safety requirements need to be highlighted when addressing core requirements from the product backlog (so that they can be addressed as an integral part of the implementation of these

Recommendations for Scrum

- A second product backlog – the safety backlog – should be maintained alongside the product backlog to explicitly store, manage and prioritise software safety requirements.
- The safety backlog should record traceability between safety requirements and system hazards
- The safety backlog should record traceability between safety requirements and ‘core’ product backlog requirements (e.g. between a safety constraint and the function to which it relates)
- The safety backlog is initially populated as a result of (deductive, ‘top down’) system level safety analysis performed prior to the software development commencing (i.e. analysis that identifies potential software contributions to system level hazards).
- The safety backlog should also be populated through

performing (inductive, ‘bottom up’) hazard analysis (e.g. functional failure analysis) on the core product backlog requirements.

- Both the system level safety analysis, and the software hazard analysis should involve the following stakeholders: the system team (as ‘customer’ to the software team), the system customer (e.g. system operator), competent safety engineers capable of carrying out the analyses, the safety regulator (as safety ‘customer’) potentially supported by addition independent and internal safety reviewers (as proxies for the regulator).
- Maintenance of the safety backlog is the responsibility of a nominated safety team member from within the software team. This team member must ensure that he or she fully understand the source and rationale behind each of the requirements in the safety backlog. (This may involve discussion with the stakeholders involved in the original safety and hazard analyses.)
- When product requirements are chosen for a given sprint, the related safety requirements from the safety product backlog should be identified, discussed and explained (by the nominated safety team member) at the beginning of the sprint

3.2 Software Safety Assurance Principle 2: The intent of the software safety requirements shall be maintained throughout requirements decomposition

As the software development lifecycle progresses, the requirements and design are progressively developed and a more detailed software design is created. Having established complete and correct software safety requirements at the highest (most abstract) level of design, it must be possible to demonstrate that the intent of those requirements is maintained as the software safety requirements are developed, reinterpreted, allocated and decomposed.

Challenges posed

- If software safety requirements are reinterpreted, allocated or decomposed during development, the new safety requirements that emerge should be documented
- If software safety requirements are reinterpreted, allocated or decomposed during development, traceability must be maintained between the original safety requirement and the new requirements or design that emerges.
- When new requirements or design emerges as a response to the original requirement they must be reviewed to check whether they fully addresses the intent of the original safety requirement.
- Justification must be maintained as to how the new requirements and design addresses the original safety requirement.

Recommendations for Scrum

- If software safety requirements are reinterpreted, allocated or decomposed during a sprint, the new safety requirements that emerge should be documented in the safety product backlog, with traceability to the original safety requirement
- As part of sprint review, those responsible for developing the original safety requirement must present the design (or refined requirements) solution proposed and the new safety requirements that have emerged.
- The sprint review should attempt to check whether the proposed solution fully addresses the intent of the original safety requirement.
- Independent stakeholders are required as part of this review. It should involve the nominated safety team member and

potentially some or all of the following: the system team (as ‘customer’ to the software team), the system customer (e.g. system operator), a suitable internal safety reviewer (as proxy for the regulator’s position).

- If it is determined that more time is required to assess the adequacy of the proposed solution (than the sprint review allows) then a more thorough review will need to be allocated as an activity to the following sprint. Whilst this activity is taking place, ideally further implementation of the solution should be avoided (i.e. other requirements from the backlogs should be prioritized)
- The decomposition of the original safety requirement into the new safety requirements, and the justification as to the adequacy of this decomposition should be documented in the assurance case by the nominated safety team member. This should take place in the sprint immediately following the review and will be presented as during the following sprint review).

3.3 Software Safety Assurance Principle 3: Software safety requirements shall be satisfied

Ultimately, it is necessary to demonstrate that any safety requirements allocated to software have been satisfied (i.e. present evidence to establish whether the required behaviour will occur in operation). It is important to present evidence that shows the satisfaction of safety requirements under anticipated operating conditions.

Challenges posed

- Verification evidence must be presented to demonstrate the satisfaction of any safety requirements claimed to be implemented. Explicit traceability must be maintained between verification evidence and implemented safety requirements.
- Verification evidence generated must be reviewed to check whether it fully addresses the intent of the safety requirement.
- Justification must be maintained as to how the verification evidence demonstrates satisfaction of the safety requirement.

Recommendations for Scrum

- Daily scrum meetings can be used to provide an early indicator for monitoring progress towards safety requirements satisfaction.
- As part of sprint review, those responsible for implementation of a safety requirement must present the solution and explain how the intent of the safety requirement is fully satisfied.
- Sprint review should attempt to check whether the implementation fully addresses the intent of the original safety requirement, and review the verification evidence presented.
- Independent stakeholders are required as part of this review. It should involve the nominated safety team member and potentially some or all of the following: the system team (as ‘customer’ to the software team), the system customer (e.g. system operator), a suitable internal safety reviewer (as proxy for the regulator’s position).
- After initial inspection in the sprint review if it is determined that more verification evidence is required (e.g. to satisfy the verification criteria of a safety standard) then further verification will need to be allocated as an activity to the following sprint. It may be necessary for this verification activity to be performed by independent team members or teams (according to Principle 4+1).
- The traceability of safety requirements to the verification evidence, and the justification as to the adequacy of the

evidence set should be documented in the assurance case [9] by the nominated safety team member. This should take place in the sprint immediately following the review and will be presented during the following sprint review.

3.4 Software Safety Assurance Principle 4: Hazardous behaviour of the software has been identified and mitigated

Whereas Principle 2 is concerned with maintaining the intent of safety requirements in the presence of increasing design commitment, Principle 4 is concerned with the potential undesirable and unintended consequences of design and implementation decisions. Principle 2 is concerned with whether lower levels of requirements and implementation do what is required (intended). Principle 4 is concerned with whether the design and implementation does anything else that is considered unsafe. These potentially emergent hazardous behaviours could firstly result from design decisions that have unintended hazardous side effects. Secondly, they can also result from implementation (process execution) errors during the software development process – e.g. modelling errors, coding errors, and tool-use errors. It is necessary to ensure that assurance effort has been targeted at attempting to reveal.

Challenges posed

- Unintended hazardous behaviour can emerge as a result of the design and implementation of both safety and core requirements and therefore has to be identified in both cases.
- Where new hazardous behaviour is identified, new safety requirements (to manage this behaviour) must be defined.
- Identifying hazardous behaviour can require system-level domain expertise from outside of the software development team.

Recommendations for Scrum

- At the planning stage of a Scrum development the chosen modelling approach, implementation language, development environment and assessment tools should be documented. Hazard analysis should be conducted on the chosen languages, processes and tools to identify the potential for the introduction of implementation errors. Where the potential for error is identified, it is necessary to provide evidence for non-introduction of error (e.g. tool qualification), controls for the minimization of error introduction (e.g. process guides) or means of demonstrating the absence of introduced errors (e.g. static analysis tools). Justification of the chosen strategies must be documented in the assurance case.
- Daily scrum meetings can be used to provide an early opportunity to identify unintended side-effects emerging from a chosen implementation approach.
- (Inductive, ‘bottom up’) Hazard analysis should be performed as part of sprint review in order to identify hazardous side effects from the design and implementation activity undertaken during the sprint. This hazard analysis should include a check of the implementation of the chosen strategy for dealing with implementation error introduction. This analysis should be performed in the context of the existing identified system level-hazards, but be open to the possibility of identifying new hazards.
- Independent stakeholders are required as part of this hazard analysis. It should involve the nominated safety team member and potentially some or all of the following: the system team (as ‘customer’ to the software team), the system customer (e.g. system operator), a suitable internal safety reviewer (as proxy for the regulator’s position).
- If it is determined that more time is required for the hazard

analysis (than the sprint review allows) then a more thorough review will need to be allocated as an activity to the following sprint. Whilst this activity is taking place, ideally further implementation of the solution should be avoided (i.e. other requirements from the backlogs should be prioritized)

- Where new hazardous behaviours (i.e. new software level contributions to system-level hazards) are identified, new safety requirements (to manage these behaviours) must be added to the safety backlog.
- Where the sprint activity and subsequent review relates to the implementation of safety requirements, the decomposition of the original safety requirement into the new safety requirements, and the justification of the identification and mitigation (through new safety requirements) of hazardous behavior should be documented in the assurance case by the nominated safety team member. This should take place in the sprint immediately following the review and will be presented as during the following sprint review).
- Where the sprint activity and subsequent review relates to the implementation of core (i.e. non-safety specific) requirements, and new hazardous behavior is identified, and new safety requirements defined, the assurance case needs to be extended to include the new safety requirement.

3.5 Software Safety Assurance Principle 4+1: The confidence established in addressing the software safety principles shall be commensurate to the contribution of the software to system risk

This principle is expressed as ‘+1’, rather than principle number 5 because it underlies the implementation of the first 4 principles. Perfect assurance of the other four principles is unachievable. For example, it is impossible to prove that all hazards have been identified, and that all the necessary corresponding safety requirements have been identified. Consequently, we have to consider how to much effort to expend in addressing the first four principles, and how much evidence to generate. We have to decide upon a sufficient level of evidence to present. This principle states that the level of evidence needs to be proportional to the level of risk associated with the software in question. For a highly critical software-intensive system, the level of confidence in addressing the first four principles needs to

Challenges posed:

- To implement this principle it is necessary to be able to identify the criticality of any software safety requirement
- Implementing this principle will affect the amount of effort required, and the level of argument and evidence presented in the assurance case, to provide assurance of the first four principles. This can affect the length of sprints, whether assurance activities take place alongside implementation activities within sprints, or need their own sprint cycles to complete. This can also affect the type of stakeholder involvement required (e.g. whether independent verification or assessment is required, or when customer stakeholder involvement is sought). It will also influence the level of rigour in modelling and implementation techniques, and the level of qualification required in tools used within the process.
- Different standards have different approaches to defining required levels of assurance: IEC61508 sets a Safety Integrity Level according to the probability delta in risk reduction; DO-178C places more focus on severity; ISO 26262 incorporates the concept of controllability of the vehicle. In addition, there are many differences in the recommended techniques and processes at different levels of criticality. If conformance to

these standards is sought, the specific requirements will need to be identified and a conformance argument established as part of the assurance case.

Recommendations for Scrum

- The safety backlog should record the criticality associated with each safety requirement. This could be expressed using the language of a particular safety standards (e.g. Safety Integrity Levels – SILs) or through risk attributes (e.g. the severity of the hazard associated with a requirement, and the degree of contribution of the requirement to the hazard).
- Safety backlog item criticality can be used within sprint planning to help manage project risk. For example, satisfactory implementation of high criticality safety requirements may be considered a project risk that should not be left until late sprint releases.
- Before the Scrum development commences, it is necessary to decide how the assurance strategy will be moderated according to criticality. In particular, how the chosen approach to representing criticality will influence the following:
 - o The amount and type of effort, stakeholder engagement, and assurance evidence required for the initial population of the safety backlog
 - o Whether a dedicated safety team member is required within the software development team
 - o The extent to which (system, regulator) customers are expected to be involved in the activities described in the first four principles.
 - o The amount and type of effort, stakeholder engagement, and assurance evidence required in reviewing safety requirements development during sprints (i.e. checking safety requirements validity)
 - o The amount and type of effort, stakeholder engagement, and assurance evidence required in demonstrating satisfaction of safety requirements
 - o The amount and type of effort, stakeholder engagement, and assurance evidence required in checking for implementation introduced errors and performing hazard analysis as part of the sprint cycles.
- The criticality of chosen requirements for the sprint (or requirements related to those chosen) should be highlighted at the beginning of the sprint. In particular, it should be highlighted where independence cannot be demonstrated between the implementation of requirements of differing criticality, the higher criticality should be used.

5. Next Steps

Based on literature review and the results of our practitioner survey [3,4] we developed the recommendations presented in the previous section. However, these recommendations need to be further evaluated. We intend to conduct semi-structured interviews with both agile and safety practitioners to review the described challenges and recommendations. Beyond these interviews, it is necessary to demonstrate how these recommendations can be applied through illustrative case study examples, and further peer review of these worked examples. The results of the first set of interviews will be published after interviews have been conducted.

6. Summary

Our proposed recommendation of 4+1 principles to Scrum process stems from the lack of a clear vision as to how to adapt agile methods to the demands of safety critical systems assurance.

This paper has summarized the 4+1 principles of software safety assurance and, discussed the challenges posed when applying 4+1 principles to Scrum, and makes initial recommendations for how the principles can be accommodation within a Scrum development.

We believe that feedback from the proposed recommendation from safety and agile practitioners will help us to understand the effective use of 4+1 principles within Scrum.

7. REFERENCES

- [1] Kenneth S. Rubin. 2012. Essential Scrum: A Practical Guide to the Most Popular Agile Process (1st ed.). Addison-Wesley Professional.
- [2] Kelly, Tim. "Software Certification: Where is Confidence Won and Lost?" Addressing Systems Safety Challenges, T. Anderson, C. Dale (Eds), Safety Critical Systems Club (2014).
- [3] Osama Doss, Tim Kelly "Assurance Case Integration with An Agile Development Method In: XP 2015, LNBIP 212, pp. 347–349, 2015.
- [4] Osama Doss, and Tim Kelly. "Challenges and Opportunities in Agile Development in Safety Critical Systems – A Survey". In: Agile methods applied to development and certification of safety-critical software Workshop, XP 2015, Helsinki, Finland, 2015.
- [5] Kelly, T.; Weaver, R.: The Goal Structuring Notation – A Safety Argument Notation. In: Proceedings of the International Conference on Dependable Systems and Networks – Workshops on Assurance Cases, Florence, Italy (2004)
- [6] Bruce Douglass. "Real-Time Agility" June 2009
- [7] Agile manifesto (2001). <http://agilemanifesto.org>.
- [8] Bruce Douglass. "Agile Systems Engineering" Sept. 2015.
- [9] Hawkins, R., Habli, I., Kolovos, D., Paige, R., and Kelly, T.: 'Weaving an Assurance Case from Design: A Model-Based Approach', in Editor (Ed.)^(Eds.): 'Book Weaving an Assurance Case from Design: A Model-Based Approach' (2015, edn.), pp. 110-117